

AST 5765/4762: Final
Due 12/1/2025, 11:59pm

Overview: In this Final project you will measure the radius of an exoplanet using exoplanet transit data. In particular, you will use data from the Spitzer Space Telescope program 30825. The program observed a transit of planet HD189733b at 8 μm and was led by Harvard University Prof. David Charbonneau and his (then) graduate student Heather Knutson (now a professor at Caltech). You will search for bad pixels and perform photometry on the pre-processed data frames provided by the Spitzer Science Center. These frames are already flat-fielded, debiased, darkened, etc. You will measure the transit depth from the resulting light curve by finding the ratio of in- and out-of-eclipse data. Finally, you will calculate a radius for the planet by scaling with the stellar radius.

Ground rules: Your project should follow the same organization as the homework assignments. Your code and functions should be documented following the standard templates. Your code should record in comments the why you chose a parameter or method, and it should be appropriately indented and spaced for easy readability. Your variables should be sensibly named. Your code should save and/or print all plots, tables, and numbers that you will use in your report/paper (see below for a clarification of what you need to hand in). Print step numbers and give a few words (and units, where applicable) to explain each printed item. If you use additional plots or tables to help you choose or verify methods and parameters, including those plots in your electronic output will help justify your choices. You can refer to these in your code comments, where you make those choices.

As a good scientist, make sure to think critically of your results! While you have time, try to fix your bugs, keeping a detailed log of where the problem might lie and what you try to do to fix it. If you don't find the bug by the end, mention the issue in your log and what other steps you would take to fix it.

The report/paper: The final report should be a report (4672) or paper (5765)--see below for 4762 vs 5765 styling requirements-- in the format of the Astrophysical Journal. Examples and format guidelines can be found on the journal's web site and in our [Overleaf](#) example. You should use LaTeX to write your report. At the end, you will produce a PDF file of your report/paper and include it in your project directory. The report/paper should be as short as possible. You must use formal English. Spelling, grammar, and punctuation all count, as does a clear and logical flow of information. Use the active voice where possible and *be concise*!

For 4762: You need to write up your results in the form of a report. You can use single column format for your report. Aim for ~6-7 report pages (not counting Abstract and References). You *will be* judged on the writing style of your report.

Unlike a regular research article, your report must include a derivation of the expression(s) used to calculate the uncertainties you report (make sure to include it in the appropriate section though). Be sure to derive a single expression that has your observables as variables, and then apply the error propagation equation to it.

For this section the Final project will be graded as follows:

Report style/format will be: 5% of your grade.

For the rest 95%:

Project execution: 40% of your grade.

Project results: 60% of your grade.

Note: You may also write it as an ApJ paper, following the 5765 rules, for 10 bonus points (if done correctly).

For **5765**: You need to write a proper PSJ/ApJ paper using the two-column format. Aim for 4-7 journal pages (not counting References). Your abstract should follow the ApJ rules and not be longer than 250 words!

As you write a proper paper, unlike 4762, you will need to write your derivation of the expression(s) used to calculate the uncertainties in your log or in your code as comments and not in the paper (this is assumed to be trivial for your fellow scientists, so it will never be included in your papers unless you discovered a new method of doing it or something similar). Be sure to derive a single expression that has your observables as variables, and then apply the error propagation equation to it.

As part of the project, you will do a AAS/DPS style 10-15 min presentation on your paper (or if you prefer, an actual research project you work on at the moment). The presentations will take place on **12/3 1:00-3:50pm**. Students will present in alphabetical order.

For this section the Final project will be graded as follows:

Paper style/format will be: 20% of your grade.

Presentation: 20% of your grade

For the rest 60%:

Project execution: 40% of your grade.

Project results: 60% of your grade.

The following sections are suggested for your report/paper, but you can modify them as appropriate for your case:

1. Title (usually generic, serious, not flashy)
2. Author, affiliation (you, University of Central Florida)
3. Abstract

4. Introduction. This should include at least a few background facts about your target and introduce the topic you are working on, with a few references to papers in a reviewed journal (search on ADS and/or Simbad; see our Paper writing lecture too)
5. Observations (include a table that summarizes the data)
6. Data Analysis
 - (a) Frame Corrections (include a figure with raw and processed data)
 - (b) Measurements (name after the actual measurement, e.g., “Photometry”)
7. Results (include tables and figures as needed)
8. Conclusions
9. References (at least 1 on topic, 1 for methods, follow ApJ style)

Useful resources:

1. NASA ADS: <https://ui.adsabs.harvard.edu/>
2. ArXiv: <https://arxiv.org/>
3. Spitzer IRAC handbook
<https://irsa.ipac.caltech.edu/data/SPITZER/docs/irac/iracinstrumenthandbook/>
4. Our LaTeX report/paper guide: <https://www.overleaf.com/read/dhbgkdzprtq> For the report, make sure to change the `\documentclass[twocolumn]{aastex63}` to `\documentclass[onecolumn]{aastex63}`

The project:

1. **(10 points)** Learn about the planet. Go to NASA ADS. Get and read the discovery paper and at least one other paper about it (the paper written for this dataset is a good one to choose). Write some of the key points you get from these papers about your target in your log (you will use this for your report/paper introduction). HW6 should have already given you a head start on this task.

2. **(10 points)** Familiarize yourself with the data in `Final/data/`. Locate the IRAC Handbook online and skim it.

Identify the science frames, the uncertainty frames, the raw frames, etc. Note that we will only be using the science frames: the Basic Calibrated Data (BCD, `_bcd.fits`). Figure out the naming scheme for these files. This dataset was taken in IRAC subarray mode, which takes **64 quick exposures, stacks them in a 3D data cube, writes a FITS file, and repeats**. The directory does not contain all the files in the dataset, but only the BCD files from the time near transit.

For reference, the directory called `full_set` contains one full set of files, so you can see what professional astronomers get when they download data. One could also download the raw data, of course. The BCD frames resulted from processing the raw frames through Spitzer's calibration software.

Mention a few things you learned about IRAC in your log.

3. **(10 points)** Start your Python script with a section where you define your variables. Come back here and add whatever constants you will need at the start of the code, as you encounter them in the process of the project. Here, you should also define the variables you will use to index the frame parameters array, so that you never index that array with constants (i.e., use `mydata[n1:n2]` and never `mydata[1:5]`).

Functions should be defined in one or more separate files, as appropriate, and need to be documented with a proper docstring.

Print the directory name where you have the data saved.

4. **(20 points total)** Write a routine that reads the data and extracts header information at the same time. By the end, your routine should be able to:

(a) **(3 points)** Read the first data block (BCD) and get any information you need from it, such as the image cube size. It should also print the number of frames in a given subarray set.

(b) **(2 points)** Allocate a 3D data cube and a 2D frame-parameter array. The latter will have one row per image. Each row will contain information about one frame, such as the photometry results. Another piece of information you will need is, from the image header, the exact time each image was taken (you will need this for plotting the transit light curve). You may need others later on. Reading through the rest of the Project will give you a better idea about what you will need to add to this array (and thus its dimensions will change as you progress through the Project). As you work and find them, come back to the code in this step and add columns in the frame parameter array. Fill in the columns as you read data, below. Print the shape of each array.

(c) **(7 points total)** Make a function that loops over the images starting at a given subarray set number and continues for a given number of sets (print these), and:

i. **(1 point)** Reads the data and header.

ii. **(2 points)** Puts the science data into the 3D data cube (remember that you are inserting them in blocks of 64 frames!)

iii. **(4 points)** Finds the observation time in the header and puts it in the frame parameters array along with the frame number. The function then uses the frame interval to calculate the timing of the subsequent frames in the set. Also populate the frame parameters array with any other header values you may need (you may discover these as you go).

In the header, FRAMTIME is the interval between images in one set. All of the time keywords (DATE OBS, MJD OBS, UTCS OBS, SCLK OBS) refer to the start of the first frame in a set. Be careful about units and precision in picking one. EXPTIME is the duration of one exposure, with the difference between that and FRAMTIME being the time to transmit the data from the readout electronics into memory and reset the array for the next integration. So, take half of EXPTIME and add it to a time keyword (be careful of units) to get the mid-time of the first frame. The subsequent images in the set are separated by FRAMTIME. There is a longer gap between sets, to move the image set from the camera to the spacecraft memory.

Every 10th file, print the filename and its DATE OBS.

5. **(10 points)** Run the function, starting at subarray set 0187 and ending at set 0686 (inclusive). Look at the results. Look at a few frames (plot them!) in detail. Skim the rest of the images too. Check that you do not have blank frames at the end of your data array, etc. You may “protect” this check inside a conditional so that it only runs when you set a variable to True by hand.

Write what you saw in your log.

6. **(5 points)** Make a Boolean mask array that is the same shape as the data array. A value of True indicates a good pixel and a value of False indicates a bad pixel. Initialize it to True. From this point forward, whenever you modify the mask array or data array, set all flagged pixels to 0 in the data array so that you are not taking bad pixels into account.

7. **(5 points)** Find the median value in each frame. Record it in the frame parameters array as the median background and subtract it from each frame.

Plot the median background vs. frame number.

8. **(20 points)** Use a sigma-rejection routine. You may use the code you wrote for an early homework assignment for this, or the astropy version but note that in either case you need to produce the Boolean array (see below) to get the points for this question.

For a given data set (see Q9), the function should find the median and standard deviation from the median. Flag as bad anything more than, e.g., 5σ from the median. Then, iterate this again, using only the unflagged (“good”) numbers to calculate the median and standard deviation in the second iteration. The function should then return a Boolean array that is True for good pixels and False for bad pixels.

9. **(5 points)** Find bad pixels. This can be done with three nested loops: an outer loop that steps by jumps of 64 frames over the dataset and two inner loops that go over y and x. Extract each set of 64 pixels at a given y, x location in each subarray set, call your sigma-rejection routine on each set, and insert the resulting 64-item Boolean array into your mask array.

Plot the number of bad pixels vs. frame number.

EXTRA CREDIT (10 points): Do it with no loops (note: that the code you use from 8 to here might need changing for this!).

10. **(10 points)** Find the approximate location of the center of the star in the first image. Fit a 2D Gaussian to a small region around that point and record the center position in the frame parameters array.

Loop over all the images, using the fitted position of the previous frame as the guess position for the current frame.

Plot each of the x and y position data sets vs. frame number on a single plot with two stacked panels.

11. (10 points) Mask the fainter companion star in each image. It will be at a constant offset from the program star. Calculate its offset from your program star using a mean or median image from the whole data set (to get a really good estimate), apply this offset to each of the centers you just calculated, and mask an appropriate disk to remove the companion star for each frame. Print the y and x offsets between the two stars.

12. (20 points) Write an aperture photometry function that:

- accepts the data, a mask, radii, and a center for photometry;
- applies the mask to the sky before computing the sky level;
- does the photometry; and
- reports the results and number of bad pixels in the aperture.

You may use your functions from past homework. You do not have to use a sub-image, since the Spitzer images are only 32×32. If you do, be careful that the sub-images are the same size and that, even if the annulus extends off the edge of the array, the sub-image just goes to the edge. There will be outlier frames when the spacecraft's reaction wheels adjust their spin rates, in which the star momentarily moves and recovers. The annulus may go off the frame then, and you don't want your routine to crash in that case by calculating an image slice that is off the edges of the array.

The function should use the appropriate steps to calculate the total bad pixels in the aperture, calculate the total flux in the aperture (aka do the photometry) and return a 1D array containing the stellar flux, the average sky, and the number of bad pixels in the aperture.

An example of how this might be done is:

- Use `disk` twice to make a mask for the sky annulus.
- Apply the bad-pixel mask for the sub-image to the sky annulus so that only the good pixels that are in the sky annulus are True.
- Calculate the average sky pixel in the annulus. Do not use any masked pixels in your sky estimate.
- Subtract this value from each pixel in the sub-image.
- Use `disk` to make a mask for the photometry aperture. Do not apply the bad-pixel mask!
- Calculate the total flux in the aperture.
- Calculate the total bad pixels in the aperture.

13. (5 points) Loop over all images and call your function to do aperture photometry. Store the aperture counts, average sky level, and number of bad pixels in the aperture in the frame parameters array.

14. **(20 points)** Check the photometric outputs by plotting just the photometry for the good frames (frames with no bad pixels in the aperture). Do you see the transit? Are there many outliers? If so, consider ignoring them, or, you may flag them as bad by putting a -1 in the frame parameters entry for number of bad pixels in the aperture. Justify each elimination with a sentence fragment in your code, such as “very high background,” “large star motion,” etc. You would want to do these as classes of eliminations, of course, not frame-by-frame.

You may not need to eliminate (m)any outliers, however (why?). Also, look at the sky, x and y centers, and number of bad pixels. Discuss in your report/paper what you see (if anything), and what you did about it. Do you have to do much, or are these frames already eliminated for some reason? Finally, inspect the two or three frames with the largest numbers of bad pixels. Describe what appears in each in the log and your paper, and what you did about it.

15. **(10 points)** Bin the data into a few hundred bins and plot the bin averages. Adjust the number of bins until you like the plot. The transit is small, a few percent of the star’s brightness, so you will have to zoom in. You should see a slight overall ramp in the data. Make a publication ready plot of the data and save it as a PDF.

16. **(4 points)** Manually identify four frames in the unbinned data where the following happens:

- (a) first contact (start of transit),
- (b) second contact (planet fully on stellar disk),
- (c) third contact (start of egress), and
- (d) fourth contact (end of transit).

Print these numbers.

17. **(16 points)** Fit a parabola to the good flux vs. time data outside of transit. Evaluate it at each frame time (all frames, not just good frames). Divide the aperture photometry by these values for each frame to normalize it to this ramp model. It won’t be perfect, but it should improve it some.

Make a publication ready plot of the data and save it as a PDF.

18. **(10 points)** Find a flat run of 100 or more normalized frames. For example a set just after the transit (avoiding long-period ripples). Calculate the standard deviation of the photometry in this set of frames. What is your S/N per frame?

19. **(10 points)** Using your uncertainties, and taking averages of the good out-of-transit and in-transit data to find the best estimate of the normalized flux for each, calculate the ratio of the planet cross-sectional area to stellar cross-sectional area (and, of course, its uncertainty). Ignore bad frames and frames from when the star’s limb cuts through the planet.

Print all of these values, including the averages.

20. (**10 points**) Look up the radius of the star (including uncertainty) in a refereed article and calculate the radius of the planet (see also HW6). Print it. Remember that information like this should be included in your report/paper, with the appropriate references too!

21. Write your report/paper! Remember, this is 5% or 20% of your grade!

22. (**10 points**) Include a copy of your class log file in your handin. Don't forget to include all information about your Git commits and the PDF file for the report/paper in your handin. Turn in to Webcourses as a .tar.gz!

Happy winter break!